

An Extensive Study on Adversarial Attack against Pre-trained Models of Code

Xiaohu Du^{*†}
Huazhong University of Science
and Technology, China
xhdu@hust.edu.cn

Ming Wen^{*†‡}
Huazhong University of Science
and Technology, China
mwena@hust.edu.cn

Zichao Wei^{*†}
Huazhong University of Science
and Technology, China
u201911736@hust.edu.cn

Shangwen Wang
National University of Defense
Technology, China
wangshangwen13@nudt.edu.cn

Hai Jin^{*§}
Huazhong University of Science
and Technology, China
hjin@hust.edu.cn

ABSTRACT

Transformer-based *pre-trained models of code* (PTMC) have been widely utilized and have achieved state-of-the-art performance in many mission-critical applications. However, they can be vulnerable to adversarial attacks through identifier substitution or coding style transformation, which can significantly degrade accuracy and may further incur security concerns. Although several approaches have been proposed to generate adversarial examples for PTMC, the effectiveness and efficiency of such approaches, especially on different code intelligence tasks, has not been well understood. To bridge this gap, this study systematically analyzes five state-of-the-art adversarial attack approaches from three perspectives: effectiveness, efficiency, and the quality of generated examples. The results show that none of the five approaches balances all these perspectives. Particularly, approaches with a high attack success rate tend to be time-consuming; the adversarial code they generate often lack naturalness, and vice versa. To address this limitation, we explore the impact of perturbing identifiers under different contexts and find that identifier substitution within `for` and `if` statements is the most effective. Based on these findings, we propose a new approach that prioritizes different types of statements for various tasks and further utilizes beam search to generate adversarial examples. Evaluation results show that it outperforms the state-of-the-art ALERT in terms of both effectiveness and efficiency while preserving the naturalness of the generated adversarial examples.

^{*}National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, HUST, Wuhan, 430074, China

[†]Hubei Engineering Research Center on Big Data Security, Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, HUST, Wuhan, 430074, China

[‡]Corresponding author

[§]Cluster and Grid Computing Lab, School of Computer Science and Technology, HUST, Wuhan, 430074, China

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEC/FSE '23, December 3–9, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0327-0/23/12...\$15.00

<https://doi.org/10.1145/3611643.3616356>

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → **Neural networks**.

KEYWORDS

Adversarial Attack, Pre-Trained Model, Deep Learning

ACM Reference Format:

Xiaohu Du, Ming Wen, Zichao Wei, Shangwen Wang, and Hai Jin. 2023. An Extensive Study on Adversarial Attack against Pre-trained Models of Code. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*, December 3–9, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3611643.3616356>

1 INTRODUCTION

Given the rapid development of *deep learning* (DL), many researchers have applied DL techniques in various programming language processing tasks with promising results achieved over recent years, and such a trend continuously rises. The recently-proposed Transformer architecture [28], which mainly employs the self-attention mechanism, has shown promising results on dealing with the long range dependency problem, which is a critical challenge for traditional sequence models such as the Recurrent Neural Network. Therefore, a number of state-of-the-art DL models are designed based on such an architecture, one category of which is the *pre-trained models of code* (PTMCs), such as CodeBERT [3] and CodeGPT [14].

Via utilizing the pre-training techniques, domain knowledge in the large-scale publicly-available code repositories can be learned by PTMCs, which can be further leveraged on downstream tasks such as vulnerability detection, clone detection, and code summarization [2, 3, 14]. Unfortunately, recent studies have shown that similar to conventional deep learning models in the domains of computer vision and natural language processing, PTMCs can also generate totally different results given two semantically-identical input programs, one of which (a.k.a. the *adversarial example*) is generated by performing certain semantic-preserving transformations to the other [7, 13, 17, 20, 34, 38, 40, 43]. This is devastating considering that PTMCs have been deployed to a wide range of mission-critical applications such as vulnerability detection [34, 39]. Specifically, an attacker may easily generate an adversarial example that retains the vulnerability while misleading the PTMC to label it as “non-vulnerable”.

One potential way to alleviate such a threat is adversarial re-training, where the models under attack are continuously trained with the generated adversarial examples to enhance the robustness [34, 36]. Therefore, over the years, a number of adversarial attack approaches have been proposed aiming to automatically generate adversarial examples [7, 13, 17, 20, 34, 38, 40, 43]. Existing adversarial attack approaches differ in terms of various design aspects. First, at a high level, the semantic-preserving transformation can be performed at both the **token level** (e.g., by identifier substitution [17]) and the **statement level** (e.g., by adding dead code [36]). Second, even if certain approaches are designed at the same granularity (e.g., identifier substitution), there also exists multiple choices when determining how and what identifiers to be replaced: by random selection [38] or via pre-defined heuristics [34]. Such design aspects may significantly affect the effectiveness of the attack approaches. For instance, Yang *et al.* [34] showed that certain pre-defined heuristics could outperform random selection on generating new identifier names.

Although huge efforts have been made towards advancing adversarial attacks targeting for PTMCs, the performance of existing techniques has not been systematically evaluated and compared. Little is known about their advantages and disadvantages. There is thus an urgent need for a comprehensive empirical study comparing and analyzing the effectiveness of the *state-of-the-art* (SOTA) adversarial attacks targeting PTMCs. In particular, how is the effectiveness and efficiency of the SOTA approaches with respect to various PTMCs? How well do different approaches generalize across various code intelligence tasks? Most importantly, how is the quality of the adversarial examples generated by different approaches? If the quality is extremely low, the practical usefulness can be compromised since they can be easily perceived by developers. Additionally, it is unclear how the context of code perturbations affects the effectiveness of current attack approaches. Understanding such problems is important to guide future researches in this field.

To fill this gap, in this study, we perform an extensive study on existing SOTA adversarial attack approaches against PTMCs. Specifically, we utilize five SOTA adversarial attack approaches to attack three widely-used PTMCs (e.g., CodeBERT [3], CodeGPT [14], and PLBART [2]). Our evaluation is performed on three well-studied code intelligence tasks, including one generation task (i.e., code summarization) and two understanding tasks (i.e., vulnerability detection and code clone detection). Through extensive evaluations and comparisons, our study makes several interesting findings: (1) PTMCs can be easily attacked under all the three tasks and they are relatively less robust on the generation tasks compared with understanding tasks; (2) There is a trade-off between the effectiveness and efficiency for the adversarial attacks: the attack approach with the highest success rate usually queries PTMCs for the most times; (3) The quality of adversarial examples is heavily influenced by the identifier substitution strategy. Identifiers predicted with context-aware information produce the highest quality examples that are very similar to the original code, followed by a cosine similarity-based substitution strategy. On the other hand, random substitution leads to the lowest quality adversarial examples; and (4) replacing identifiers under different types of statements exhibits diverse chances to generate adversarial examples successfully while

such chances differ significantly with respect to the generation and understanding tasks.

Based on our findings, we design an efficient yet effective attack approach called BeamAttack for code adversarial attack. BeamAttack separates identifiers into several groups based on the statements where they are extracted. It then iteratively selects identifiers in a prioritized manner, selecting those that are most likely to result in successful attacks, as summarized by our empirical evaluation. BeamAttack reduces the attacking costs by dividing identifiers into smaller sub-groups and prioritizing them based on the likelihood of successful attacks, rather than searching the entire identifier space. It can also reduce the risk of getting stuck in local optima, as opposed to searching each individual identifier similar to WIR [38]. The results on a total of six datasets demonstrate that our approach achieves higher attack success rates with less queries than ALERT while can preserve the naturalness of the generated adversarial examples (i.e., the generated examples bear a high resemblance to the code written by humans).

To summarize, we make the following major contributions:

- **Originality.** To our best knowledge, we perform the first extensive study on existing SOTA adversarial attacks approaches towards PTMCs under well-studied code intelligence tasks.
- **Extensive Study.** We systematically compare five state-of-the-art adversarial attack approaches from three perspectives: effectiveness, efficiency, and the quality of generated examples. Our evaluation reveals the strengths and weaknesses of existing approaches, highlights useful insights, thus paving the way for future researches in this field.
- **Improvement.** Based on our empirical findings, we exploit the differences among diverse program contexts with respect to the chances of successfully generating adversarial examples and design a simple yet effective attack approach. Our approach has demonstrated promising results via extensive evaluation.
- **Open Science.** We have released all the artefacts of our study, including the source code and experiment results, which available at: https://github.com/CGCL-codes/Attack_PTMC.

2 BACKGROUND

2.1 Pre-Trained Models of Code

PTMC can learn universal language representations on the large corpus and can avoid training a new model from scratch [6, 19]. The pre-training paradigm usually consists of two stages: pre-training and fine-tuning. In the pre-training stage, it captures generic language knowledge by employing self-supervised learning on a large unlabeled corpus. In the fine-tuning stage, the trained model can be fine-tuned for different downstream tasks. PTMC can be divided into three categories based on their architectures: encoder-only, decoder-only, and encoder-decoder models [38]. Encoder-only pre-trained models can support both the understanding and generation tasks, and the most widely used ones are CodeBERT [3] and GraphCodeBERT [5]. Decoder-only models are good at generation tasks like code completion while the adopted unidirectional architectures are less effective on understanding tasks such as clone detection [4]. CodeGPT [14] is a well-known model based on Transformer belonging to this category. Encoder-decoder models are proposed

aiming to tackle both the understanding and generation tasks, and PLBART [2] as well as CodeT5 [33] are typical ones of such models.

2.2 Adversarial Attack on Code

2.2.1 Code Processing Tasks. Following prior works [14, 38], we briefly introduce three typical code processing tasks, which involve the understanding task and generation task.

Clone Detection. It is a program understanding task aiming to detect whether two source code snippets are identical or similar.

Vulnerability Detection. It is another program understanding task whose purpose is to determine if a given code snippet contains vulnerabilities or not.

Code Summarization. It is a generation task, which aims to generate natural language texts that describe the functionality of a given code snippet.

2.2.2 Definitions. We give two definitions respectively for the understanding task and generation task. For the understanding task, a classifier $f : \mathcal{X} \rightarrow \mathcal{Y}$ is expected to predict the ground-truth label $y_{truth} \in \mathcal{Y}$ for a given code snippet $x \in \mathcal{X}$. The goal of adversarial attack is to add slight perturbations on x to generate adversarial examples x^{adv} that can mislead f . Specifically, an adversarial code example should satisfy the following three requirements: (1) Adversarial example should mislead the model: $f(x^{adv}) \neq f(x) = y_{truth}$. (2) Adversarial perturbations should ensure the code is still syntactically correct. That is, perturbations should conform to the syntax rules of the programming language. For example, for the C language, the identifiers can only contain letters, numbers, and underscores. (3) x^{adv} should be semantically equivalent to x (i.e., have exactly the same functionalities and generate the same results given a same input). For the generation task, we take the code summarization as an example. The model $f : \mathcal{X} \rightarrow \mathcal{Y}$ aims to maximize $P(y_{truth} | x)$ where a given code snippet $x \in \mathcal{X}$ and the ground-truth summary $y_{truth} \in \mathcal{Y}$. Since the output y of summarization models contains many possibilities, we cannot employ the first requirement in the understanding task to directly determine whether the attack is successful. The existing work [43] utilizes the decrease on the BLEU score to evaluate the performance of attack approaches. In this paper, we follow the existing study [38] to consider an attack successful when the BLEU score between adversarial summary and the reference summary is 0, which indicates that the adversarial summary does not match the reference summary at all. Similarly, adversarial examples in generation tasks should also meet the requirements (2) and (3) as defined in the understanding tasks.

3 STUDY DESIGN

In this section, we introduce the design of our empirical study, including the selected pre-trained models, adversarial attack approaches, and benchmark datasets. We then introduce our designed research questions and the corresponding experimental settings.

3.1 Subjects and Datasets

3.1.1 Target Models. Section 2.1 presents the SOTA PTMCs to date. For each category of PTMCs, we choose one model for evaluation as the previous report [38] indicates that they achieve very close performance. Meanwhile, there is no PTMC model that can achieve the optimum performance across different tasks and datasets (e.g., for encoder-only model, CodeBERT is better than GraphCodeBERT in

vulnerability detection while vice versa in clone detection. Similarly, for the encoder-decoder model, CodeT5 outperforms PLBART in vulnerability detection while vice versa in clone detection [38]). Therefore, we select the most popular and widely-used one (indicated by the number of citations) for each category. In particular, we select CodeBERT [3], CodeGPT [14], and PLBART [2] in our study from each category.

3.1.2 Adversarial Attack Approaches. Table 1 summarizes the state-of-the-art adversarial attack approaches published in the major conferences and journals. The approaches selected in this study are all black-box approaches since (1) white-box attacks often require to access the information of model structures and parameters which might not be easily obtained in practice, and thus attackers typically can only access the provided APIs to query the model; and (2) white-box attacks tend to be model-specific in that different models employ different structures, and thus an attack approach against a specific model cannot generalize well to other ones. However, in this study, we aim to evaluate the selected attack approach against different models under various applications. Among the listed black-box attacks, we exclude the approach proposed by Nguyen et al. [16] because it performs fake API insertion at the class level while the datasets selected to evaluate the three tasks in this study are all at the function level. Among the remaining eight black-box attacks, four different attacks [34, 38, 40, 43] can be directly reproduced and are thus selected as our study subjects. As for the remaining four approaches [7, 13, 17, 20], they perform semantic-preserving transformations at the statement level (e.g., by inserting dead code or transforming for loop into while loop). Because such approaches usually contain common and similar transformation strategies, we summarize widely-used strategies and integrate them into one approach. We briefly introduce the selected five approaches below.

MHM [40]. MHM performs iterative identifier substitution based on *Metropolis-Hastings* (M-H) sampling [15]. This attack has two main hyperparameters, the maximum number of iterations and the number of candidate identifiers per iteration. The larger the value, the higher chance the attack will be successful. Unfortunately, it will be less efficient at the same time. We set these two parameters to 100 and 30 respectively, following the original paper [40].

ACCENT [43]. ACCENT first selects K candidates for each identifier based on the cosine distance, and then selects the best identifier and candidate based on the change in scores before and after substituting the identifiers. This approach has two main hyperparameters, the number of candidate identifiers K and the number of the identifiers max that can be replaced. For a fair comparison with other attacks, we set k to 30, and cancel the parameter max which means we do not limit the number of replaced identifiers.

WIR-Random [38]. WIR-Random utilizes *Word Importance Rank* (WIR) to determine the substitution sequence of identifiers, which ranks each identifier according to the difference in the probabilities generated by the model before and after renaming the identifier to “UNK”. Then, WIR-Random sequentially replaces the sorted identifiers by randomly selecting candidates. For fair comparison with MHM, we also limit the number of candidate identifiers to 30. **ALERT [34].** ALERT utilizes context-aware identifier prediction for substitution. In particular, in terms of the identifier selection strategy, ALERT adopts two methods, the greedy algorithm and the genetic algorithm. We set the relevant hyperparameters following

Table 1: Summary of existing adversarial attack approaches (in ascending order by the publication year)

Approach	Venue	White/Black	Task	Perturbation
DAMP [36]	OOPSLA'20	White	Functionality Classification Code Completion	Random Substitution Dead-Code Insertion
MHM [43]	AAAI'20	Black	Functionality Classification	Random Substitution
Srikant <i>et al.</i> [26]	ICLR'21	White	Functionality Classification	Random Substitution Dead-Code Insertion
Rabin <i>et al.</i> [20]	IST'21	White & Black	Method Name Prediction	Random Substitution Style Transformation
Pour <i>et al.</i> [17]	ICST'21	Black	Method Name Prediction Code Captioning Code Search Code Summarization	Random Substitution Style Transformation
Nguyen <i>et al.</i> [16]	ASE'21	Black	API Recommender	Fake APIs Insertion
AVERLOC [7]	SANER'22	Black	Code Summarization	Random Substitution Style Transformation
ACCENT [43]	TOSEM'22	Black	Code Summarization	Based on Cosine Distance
ALERT [34]	ICSE'22	Black	Authorship Attribution Clone Detection Vulnerability Detection	Context Prediction
RoPGen [13]	ICSE'22	Black	Authorship Attribution	Style Transformation
CARROT [39]	TOSEM'22	White	Functionality Classification Clone Detection Vulnerability Detection	Random Substitution
WIR-Random [38]	ISSTA'22	Black	Vulnerability Detection Code Summarization	Random Substitution

the original paper, including the number of candidate identifiers (i.e., 30) and the maximum number of iterations (the larger one between $5 \times Num_i$ and 10, where Num_i denotes the number of identifiers in the code).

StyleTransfer [7, 13, 17, 20]. The idea of StyleTransfer is to perform certain transformations that do not alter the semantics of the program. In this attack, we select some common transformation strategies from existing studies including (1) randomly adding a `log` statement; (2) replacing `while` and `for` loops with each other; (3) exchanging two independent statements; (4) reordering a binary condition; (5) exchanging `switch` to `if`; (6) randomly adding a `try-catch` block; (7) randomly adding a piece of dead code; (8) switching the value of a `boolean` variable and propagating this change. Then, we apply transformations to generate N candidate examples and use them to attack the model. N is set to 500 in this study to avoid the huge overhead in the attack process.

3.1.3 Datasets. To ensure the comprehensiveness of our understanding towards the performance of existing attacks, we study three tasks: *vulnerability detection*, *clone detection* for understanding tasks, and *code summarization* for generation tasks. We select representative benchmarks to evaluate them. For clone detection, BigCloneBench [27] is a widely used clone detection benchmark that contains four main types of intra-project and inter-project clones. To better evaluate the adversarial attacks, we adopt the filtered dataset proposed by Yang *et al.* [34]. Their filtering strategies include removing unlabeled data, balancing the two labels (clones and non-clones), and making the data at a computationally friendly scale. As a result, our dataset includes 90,102 examples for training and 4,000 examples for validation and testing, respectively. For vulnerability detection, the *Open Web Application Security Project* (OWASP) Benchmark¹ is a Java test suite designed to evaluate vulnerability detection tools, and it is widely used in vulnerability detection tasks [9, 23, 25]. We adopt version 1.1 of this benchmark, which contains more data and is suitable for training models. As a result, the dataset includes 13,041 examples for training and 4,000

¹<https://owasp.org/www-project-benchmark>

examples for validation and testing, respectively. For code summarization, CodeSearchNet [11] is a widely used dataset, which includes data from six programming languages. We follow existing works [14, 38] and use the filtered Java sub-datasets for code summarization, which results in 164,923 examples for training, 5,183 for validation, and 10,955 for testing.

3.2 Evaluation Metrics

We adopt the following metrics for evaluation.

Accuracy. It is the proportion of correctly predicted instances in the test set, which is used in the task of vulnerability detection.

Precision, Recall, and F1 Score. These three metrics are used for evaluating clone detection. Precision (P) is the proportion of cloned pairs correctly predicted as cloned to all pairs predicted as cloned. Recall (R) is the proportion of cloned pairs correctly predicted as cloned to all known real cloned pairs. F1 is the harmonic mean of precision and recall and it is calculated as: $F_1 = 2 * (P * R) / (P + R)$. **BLEU-4.** BLEU is widely used to evaluate the textual similarity between the text generated in generative systems and the ground-truth. BLEU-4 [30, 38] is a variant of BLEU, where the 4 indicates that four consecutive words (4-gram) are used as the matching unit.

We fine-tune PTMCs following existing works [14, 38], and Table 2 lists the reproduced results. The results are consistent with the previously reported ones in the original paper, which indicates that the models in our experiments have been adequately fine-tuned.

Table 2: Evaluation results on pre-trained models of code

Task	VD	CD			CS
Metrics	Acc	Precision	Recall	F1	BLEU-4
CodeBERT	98.70	96.42	96.32	96.32	18.75
CodeGPT	97.45	96.55	96.52	96.52	15.36
PLBART	99.52	96.83	96.83	96.82	17.60

VD: Vulnerability Detection; CD: Clone Detection; CS: Code Summarization

3.3 Research Questions

The goal of this study is to systematically evaluate and compare the performance of the SOTA adversarial attack approaches against various PTMC under different PL tasks, including their effectiveness and efficiency. More importantly, we are also curious to know the code qualities of the generated adversarial examples since it is reported that the quality of the generated examples is of significant importance [34]. To our best knowledge, it is also the first large-scale investigation towards the quality of the adversarial examples. Besides, we also investigate whether the context of perturbed identifiers will affect the performance of existing adversarial attack approaches. We introduce our target RQs in detail as follows:

RQ1: (Attacking performance) How do existing adversarial attack approaches perform against different PTMCs under various tasks? In this RQ, we attempt to thoroughly compare the SOTA adversarial attack approaches based on two criteria [37, 39].

C1: Effectiveness. We compare the effectiveness of adversarial attacks according to the *Attack Success Rate (ASR)*, which is the percentage of code snippets on which an attack approach can successfully generate adversarial examples, given a code dataset. A higher ASR indicates a more effective attack.

C2: Efficiency. We compare the efficiency of adversarial attacks according to two metrics: (1) *Average Model Queries (AMQ)*. AMQ denotes the number of queries to the attacked model during the

generation of adversarial examples, which is positively related to the attack running time. Too many model queries will be abnormal and suspicious for the attacked party. (2) *Average Running Time (ART)*. ART is an overall metric of the efficiency of the attack approach. It is not only related to the number of model queries, but also to the perturbation strategy. For example, the genetic algorithm is more time-consuming than the greedy algorithm [8].

RQ2: (Adversarial code quality) What is the quality of adversarial examples generated by adversarial attacks? Naturalness is crucial in adversarial example generation [34], as highlighted by ACCENT [43]: people will easily argue that if the replaced identifiers are significantly different from the original ones, the summary should be different. Therefore, they use *cosine similarity* to constrain adversarial examples. According to the existing works on the evaluation of perturbation towards text [32, 41] and code [34, 43], there are two main aspects concerning the quality of the generated examples. First, the number of tokens that are replaced should be as small as possible. Second, the adversarial tokens need to be as similar as the original ones in terms of their semantics. In this study, we evaluate the former with *Identifier Change Rate (ICR)* and *Token Change Rate (TCR)*, and the latter with *Average Code Similarity (ACS)* and *Average Edit Distance (AED)*, following existing studies [32, 34, 41, 43]. The calculation of these four metrics are as follows. (1) For k adversarial examples, if there are in total m_i identifiers in the i_{th} code snippet and n_i identifiers have been changed in the adversarial examples, then ICR is evaluated as $\sum_1^k n_i / \sum_1^k m_i$; (2) Beyond the identifiers, the source code may contain other code tokens such as keywords, operators, etc. TCR is the ratio of the changed tokens in the adversarial example to the total number of tokens in the entire code. (3) We use the cosine similarity to reflect the code similarity before and after the perturbations are performed. In particular, ACS is computed based on the embeddings that vectorized from the source code by CodeBERT; (4) AED reflects the character-level token differences, which is the number of times a token needs to be edited at the character level in order to transform into another. In general, a high-quality adversarial example should preserve lower ICR, AED, and TCR while the ACS should be higher.

RQ3: (Context of perturbed identifiers) How do the contexts of the perturbed identifiers affect the adversarial attacking performance? Existing adversarial perturbations tend to treat all identifiers equally, which leads to a large search space and might also compromise the attacking efficiency. To reduce the search overhead, we aim to explore the impact of the contexts of different identifiers on the attacking results in this RQ. In particular, we regard the statements where the identifiers reside as contexts and investigate whether perturbing identifiers residing at different contexts will affect the attacking effectiveness. In this study, we select the top five statements that are commonly used in code [18] for investigation, which are *Return*, *If*, *Throw*, *Try*, and *For* statements. In addition to these types of statements, we also investigate the impact of merely modifying method names and the parameters to verify whether the models are vulnerable to such changes. **We refer to them as Method in this study.** We use ASR to observe the impact. Particularly, we choose two attacks with the highest ASR, which are MHM and WIR-Random. Finally, we use CodeBERT as the target model because it is the most studied PTMC to date.

3.4 Settings of Attacks

We use the trained models as introduced in Section 3.1.1 as the attack targets and adapt the original code of the five attack approaches in this study. In particular, we only make limited modifications on the code, specifically focusing on the data loading and a few parameters (e.g., the candidate identifiers as mentioned in Section 3.1.2), to serve for the need of processing our selected datasets. We use all the test set as the target instances (i.e., in total 4,000) for attacks on vulnerability detection and clone detection. For the code summarization task, we randomly select 4,000 examples from the test set as instances used for attacks to align with the number of the target instances used in the other two tasks. Meanwhile, it is beneficial for our study to explore the differences between the robustness of models for different tasks under the same scale of adversarial attacks. When evaluating the attack approaches based on identifier substitution, we skip source programs without identifiers. Besides, we also skip the instances that are classified incorrectly by the model to mitigate the effect of model performance. Such settings are commonly used in adversarial attacks [34, 40]. Although we exclude a small proportion of instances, our study is large-scale. In particular, we perform attacks on more than 150,000 target programs with over 100 million queries to various PTMC models.

4 EMPIRICAL RESULTS

In this section, we present the results of our empirical studies.

4.1 Attack Performance (RQ1)

4.1.1 Effectiveness. We perform experiments on the five attack approaches and measure their *Attack Success Rate (ASR)*, and the results are shown in Table 3. Generally, all the three target models can be easily attacked under the three different tasks. In particular, MHM can achieve the highest ASR (i.e., 57.83%) averaged over all the experiments, followed by WIR-Random (i.e., 38.77%). Based on the results, we make the following observations.

First, *random substitution is more effective than the other perturbation strategies*. Specifically, both MHM and WIR-Random adopt the strategy of random substitution while ALERT perturbs identifiers based on context-aware prediction. Consequently, MHM and WIR-Random outperform ALERT by 184.60% and 90.80%, respectively. Meanwhile, such outperformance can be observed for all the three tasks, which reflects that random substitution is the most effective strategy to mislead pre-trained models. On the contrary, StyleTransfer is less effective. We conjecture the behind reason is that existing trained clone detection models are more robust to various code transformation strategies. For example, the cloning method summarized by Walker *et al.* [29] includes adding/deleting code snippets and reordering statements, which is very similar to the strategies as adopted by StyleTransfer. Therefore, the clone detection model can learn sufficient code transformation features on such code clone pairs, thus being robust to StyleTransfer.

Second, *the models are more robust against adversarial attacks under the understanding tasks than the generation tasks*. In particular, we observe that the ASR of the five attacks on the code summarization model is higher than that of the clone detection and vulnerability detection. The average ASRs of clone detection and vulnerability detection are 24.39% and 22.62%, much lower than that of code summarization, which is 52.25%. Among them, MHM

Table 3: Attack success rate on pre-trained models of code

Attack Approach	MHM	ACCENT	ALERT	WIR	StyleTransfer	Avg	
CD	CodeBERT	47.13	21.58	14.48	35.00	0.42	23.72
	CodeGPT	43.90	23.55	6.59	35.43	0.27	21.95
	PLBART	45.89	51.64	9.10	30.23	0.68	27.51
VD	CodeBERT	57.12	31.10	4.23	18.17	21.97	26.52
	CodeGPT	29.68	24.95	4.75	13.11	9.04	16.31
	PLBART	25.09	35.07	17.28	22.23	25.58	25.05
CS	CodeBERT	93.80	64.85	48.58	79.84	29.97	63.41
	CodeGPT	87.11	21.17	23.50	52.90	6.71	38.28
	PLBART	90.76	53.73	54.37	62.00	14.45	55.06
Average Number	57.83	36.40	20.32	38.77	12.12		
CD:Clone Detection; VD:Vulnerability Detection; CS:Code Summarization							

CD:Clone Detection; VD:Vulnerability Detection; CS:Code Summarization

achieves an ASR over 90% on the three code summarization models on average, which shows that these models can be easily attacked under the task of code summarization, and output completely irrelevant summaries compared to their original outputs.

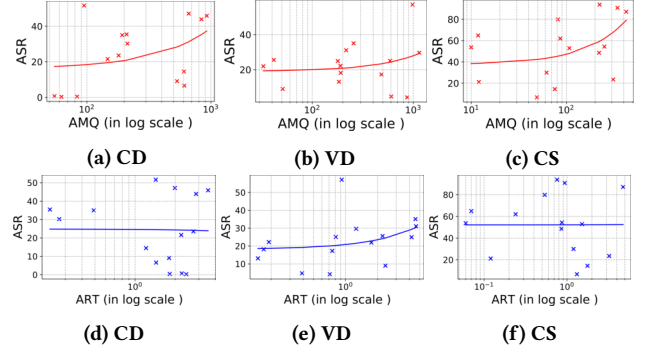
Third, *among the different pre-trained models, CodeGPT is more resistant to various attacks*. CodeGPT achieves the lowest ASR in 11/15 of the experiments (five attacks for three tasks). The average ASR over the three tasks of the five attacks on CodeGPT is 25.51% as shown in the last column of Table 3, which is lower than that of CodeBERT by 37.88% and PLBART by 35.87% respectively.

Finding 1: Pre-trained models with excellent performance can be easily misled by various adversarial attacks. In particular, random strategies are more effective; models for the generation tasks are less robust compared to understanding tasks; and CodeGPT is in general more resistant to various attacks.

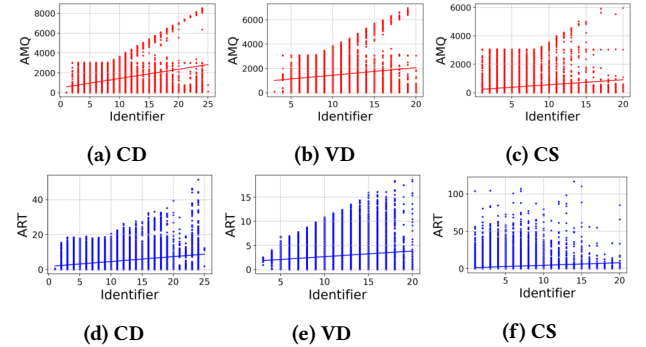
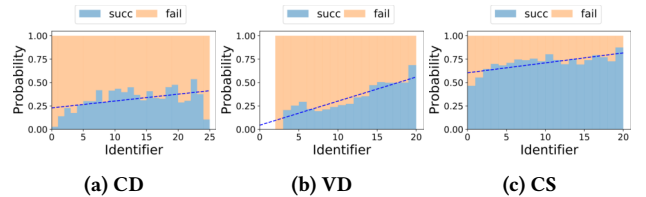
4.1.2 Efficiency. Table 4 shows the results with respect to AMQ and ART. Via analyzing these two metrics in conjunction with ASR, we make the following observations.

First, *the efficiency of different attacks varies greatly*. The average AMQ of ALERT and MHM is 1,945.69 and 1,613.90 respectively, while that of WIR-Random and ACCENT is only 212.98 and 159.87. Such differences are caused by the characteristics of the attack approaches themselves. In particular, MHM employs a large number of iterations while StyleTransfer only transfers the target code for a limited number of times to maintain the naturalness of the code. Besides, ALERT uses the genetic algorithm with multiple iterations, which tends to repeatedly replace the same identifier, while WIR-Random and ACCENT only replace identifiers sequentially according to their importance calculated by the algorithm, and they will not repeat replacing identifiers. For the same attack, the efficiency varies on different tasks as well. Specifically, the average AMQ of the five attack approaches on the three PTMCs is 1,078.80 and 1,181.77 on clone detection and vulnerability detection, but this value is 366.26 on code summarization. Further analysis reveals that it is caused by the low robustness of code summarization models (Finding 1). The high ASR of the attack approaches on code summarization models means that attacks can terminate early without performing all iterations or visiting all replaceable variables.

Second, *the number of model queries is positively correlated with the attack successful rate in general*. Figure 1 depicts the correlation between AMQ and ASR. As it reveals, attacks with a higher ASR often require a larger number of AMQ. For example, the MHM with the highest ASR has an average AMQ of 1,613.90 across all models, while the corresponding values of ACCENT and StyleTransfer with

**Figure 1: The correlation between AMQ&ART and ASR**

a lower ASR are 159.87 and 445.59 respectively. However, the running time (ART) is not necessarily positively correlated with ASR. Specifically, although StyleTransfer queries the models for less times than MHM (445.59 vs 1,613.90 on average), it takes much longer time for StyleTransfer to process the queries than MHM (14.72 mins vs 4.39). As a result, the metric ART is not positively correlated with ASR as shown in Figure 1. It is because an attack approach often contains additional time consumption besides querying the model. For instance, StyleTransfer usually spends a lot of time on code transformation.

**Figure 2: The correlation between AMQ&ART and the number of identifiers in the target program****Figure 3: The correlation between ASR and the number of identifiers in the target program**

Third, *the efficiency of various attacks is affected by the total number of identifiers that can be extracted from the program*. As shown in Figure 2, AMQ and ART always increase with the number of identifiers, and this trend holds for all the three tasks. Such a trend arises from the fact that the number of replaceable identifiers plays the fundamental role in attacks. Specifically, both ALERT and WIR utilize the greedy algorithm to iterate over all identifiers. Therefore, in the worst case, where the attack fails, its theoretical number of queries is the product of the number of identifiers and the

Table 4: Average Model Queries (AMQ) and Average Running Time (ART) on attacking CodeBERT, CodeGPT, and PLBART

Attack Approach		Average Model Queries (AMQ)					Average Running Time (ART) (min)				
		MHM	ACCENT	ALERT	WIR-Random	StyleTransfer	MHM	ACCENT	ALERT	WIR-Random	StyleTransfer
Clone Detection	CodeBERT	1,884.43	196.09	2,263.53	247.65	498.12	5.69	2.95	4.49	0.63	9.25
	CodeGPT	2,040.39	206.35	2,596.18	250.58	498.83	6.94	3.13	5.89	0.27	14.09
	PLBART	2,040.01	157.03	2,549.49	256.27	496.97	7.79	2.36	8.39	0.33	18.35
Vulnerability Detection	CodeBERT	1,877.43	235.83	2,718.57	276.30	397.33	1.75	5.09	2.02	0.24	20.00
	CodeGPT	2,497.18	228.29	2,798.29	282.98	459.41	2.72	5.33	1.55	0.23	21.42
	PLBART	2,446.50	256.31	2,592.32	277.02	382.72	3.37	4.66	3.65	0.27	20.07
Code Summarization	CodeBERT	395.01	36.49	565.63	90.31	368.71	1.38	0.21	2.30	0.61	7.19
	CodeGPT	756.67	76.15	938.39	126.64	469.58	8.29	0.80	8.78	1.83	11.80
	PLBART	587.51	46.27	488.82	109.07	438.67	1.62	0.26	1.70	0.30	10.34
Average Number		1,613.90	159.87	1,945.69	212.98	445.59	4.39	2.75	4.31	0.52	14.72

number of potential candidates. We further explore the correlation between the number of identifiers in the target program and the attack successful rate, and the results in Figure 3 show that more identifiers can in general lead to higher attack successful rates.

Finding 2: There is a trade-off between the effectiveness and efficiency for adversarial attacks. Attacking with higher successful rates often requires a larger number of model queries. Besides, the efficiency of attack is also affected by the number of identifiers in the target program.

4.2 Adversarial Code Quality (RQ2)

The above RQ demonstrates the effectiveness and efficiency of adversarial attacks against different PTMCs under various tasks. However, with the recent focus on the naturalness of the generated adversarial examples [34], a question naturally arises: which attack generates adversarial examples of higher qualities? Via analyzing the results of ICR, TCR, ACS and AED are shown in Figure 4, we make the following observations.

First, *none of the attacks can achieve the optimal performance on the three tasks in terms of naturalness*. Specifically, ALERT achieves the best in terms of ICR, ACS, and AED on average, while ACCENT is the optimal on average against TCR. The results are also different on various tasks. For instance, ALERT outperforms ACCENT on average against ICR, ACS, and AED on clone detection and vulnerability detection, but vice versa on code summarization. In general, ACCENT and ALERT outperform MHM and WIR-Random in terms of the averaged ACS, and AED on all tasks since they both consider the naturalness of adversarial examples when replacing identifiers.

Second, *effective attacks in general generate less natural adversarial examples*. The adversarial examples generated by MHM and WIR-Random with the highest ASR have the lowest ACS to the original code and the largest ICR, TCR, and AED, indicating that their adversarial code quality is generally lower. This raises the question towards the usefulness of MHM and WIR-Random in practice since an existing study points out that the adversarial example should not only cheat the model but also be natural to human judges [34]. Conversely, the adversarial codes generated by ALERT and ACCENT perform best on both ACS and AED, indicating that these adversarial code are more similar to the original code. Such results also confirm that the adversarial code are more natural than random replacement as claimed by ALERT and ACCENT. A potential reason for the high attack successful rate of MHM is that less natural perturbations may lead to *Out-of-Distribution* (OOD) examples.

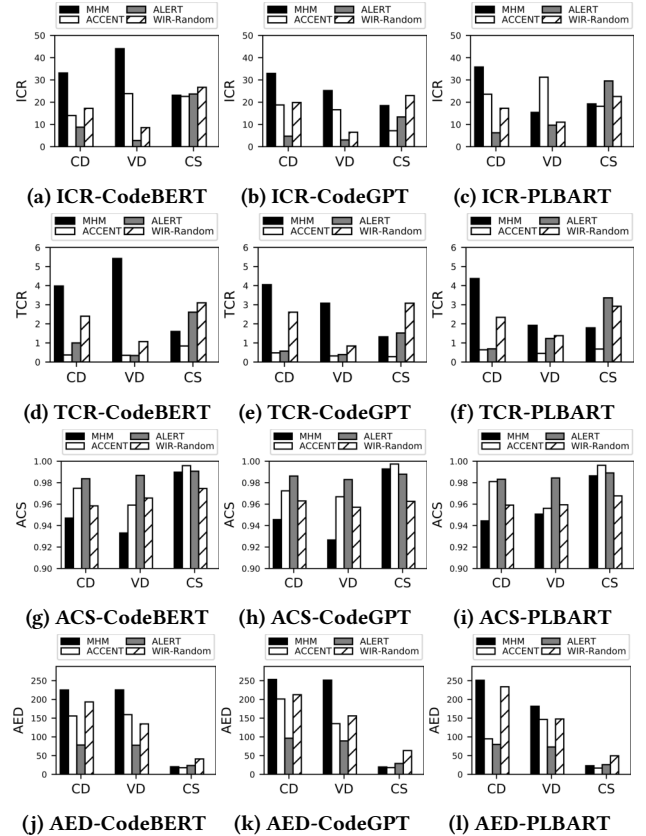


Figure 4: Comparison of ICR, TCR, ACS, and AED on attacking CodeBERT, CodeGPT, and PLBART. The lower ICR, AED, and TCR with the higher ACS indicates better performance.

Such examples could easily lead to the success of adversarial attacks because the models may not perform well on data with different distributions [22].

Third, *the adversarial examples generated by attacking CodeGPT are less similar to the original program than the other pre-trained models*. Specifically, when the target model is CodeGPT for all attacks, the average ACS is 0.9701, which is lower than 0.9716 and 0.9715 of CodeBERT and PLBART, and the average AED is 127.15, which is higher than 112.72 and 110.35 of CodeBERT and PLBART. Such differences are all significant as revealed by the Mann-Whitney U test [21] (p -value<0.05). This result also confirms Finding 1, that is, CodeGPT is more resistant to various attacks. Since ASR is negatively correlated with naturalness, the attack algorithm has

Table 5: Attack success rate for replacing identifiers under different contexts

Position	Clone Detection					Vulnerability Detection					Code Summarization				
	Total	MHM		WIR-Random		Total	MHM		WIR-Random		Total	MHM		WIR-Random	
		n=1	n=3	n=1	n=3		n=1	n=3	n=1	n=3		n=1	n=3	n=1	n=3
Method	4,000	11.01	17.90	1.92	9.91	4,000	11.16	19.81	2.36	5.70	4,000	61.32	88.81	46.84	80.81
Return	1,650	11.08	11.26	1.83	2.08	2,659	10.60	10.60	2.49	2.49	2,303	26.81	29.77	14.22	18.25
If	2,574	15.07	23.37	3.00	11.42	2,773	12.38	21.07	2.33	5.47	2,446	24.81	32.40	13.72	22.33
Throw	899	10.81	13.39	1.23	3.07	2,337	7.37	7.37	1.58	1.58	462	25.21	27.12	12.33	14.25
Try	2,858	13.78	23.89	2.30	12.77	2,501	8.41	15.24	1.93	4.33	603	22.87	30.77	11.13	20.85
For	741	16.11	26.05	3.08	14.29	398	10.98	18.40	2.97	5.64	398	26.73	35.14	15.32	29.13

to choose the suboptimal word with a longer distance among the candidate identifiers to achieve the purpose of misleading CodeGPT.

Finding 3: There is a trade-off between the effectiveness and naturalness for adversarial attacks. Specifically, effective attacks generate less natural adversarial examples. In general, substitution strategies such as context-aware identifier prediction and replacement based on cosine similarity can generate examples of higher qualities than that of random substitution.

4.3 Contexts of Perturbed Identifiers (RQ3)

The exploration of the above two RQs demonstrates that there are two factors affecting the effectiveness of adversarial attack, the search algorithm and the identifier substitution strategy. However, they both concentrate on how to change the programs, while another important perspective for adversarial attack is determining *what* identifiers should be changed. In this RQ, we investigate if perturbing identifiers under different contexts can cast significant impact on the attacking effectiveness. Via analyzing the results as shown in Table 5, we make the following observations.

A large number of instances can be successfully attacked even if only one identifier is replaced. Specifically, we limit the replaced identifiers to 1 and 3 respectively. The results show that existing techniques can still attack various models successfully, and the ASR increases as such a threshold increases. For different attack approaches, MHM is consistent with previous experiments in that ASR is higher than WIR-Random in all cases. In this paper, we explore the impact of identifiers in different statements on the models' performance. Next, we take MHM as an example to make such explorations since the ASR of MHM and WIR-Random on different statements share similar trend. To ease for presentation, we refer to the different statements by their name, such as *For*.

The results show that *the attacking effectiveness is sensitive to the identifiers under various types of statements, and such sensitivity diverges across various tasks.* Specifically, in clone detection, replacing identifiers in *For*, *If*, and *Try* is more likely to result in a successful attack. For instance, when $n=3$, the ASR of these three statements all exceed 20%, and the highest is 26.05% of *For*. The perturbations to *Method*, *Throw*, and *Return* achieve relatively low ASRs (less than 20%), the lowest of which is 11.26% of *Return*. In vulnerability detection, the first three most effective statements are *If*, *Method*, and *For*. When $n=3$, the ASR of these three statements are close to 20%, and the highest is 21.07% of *If*. *Throw* has the lowest impact (ASR=7.37%). In code summarization, *Method* dominates the attack effectiveness with an extremely high ASR of 88.81%, which is significantly higher than those of the other statements. Specifically, the ASRs of the remaining types of statements are all below 50%.

Among them, *Throw* has the lowest impact on the code summarization model, with its ASR reaching 27.12%. We further perform a case study to analyze why prioritizing statements can significantly affect the performance of adversarial attacks (see Section 6.1).

Finding 4: The context of the identifiers (e.g., where the identifiers reside) can affect the attacking effectiveness significantly, which suggests that the perturbation strategies should consider the context of identifiers aiming for more effective attacks.

5 NEW APPROACH

Our empirical investigation reveals two main challenges for adversarial attack against PTMC, which are the trade-off between effectiveness and efficiency (Finding 2) as well as that between effectiveness and naturalness (Finding 3). Both the two challenges may compromise the practical usefulness of existing adversarial attacks. Aiming to alleviate the second challenge, the state-of-the-art approach, ALERT [34], adopts a context-aware identifier substitution strategy to improve adversarial code naturalness. However, our experiment reveals that both the effectiveness and efficiency of ALERT are still limited. For instance, it only achieves an average ASR of 20.32% on the three tasks.

Our tool aims to enhance both the effectiveness and efficiency while guaranteeing the naturalness, and the novelty of which is mainly embodied in the following two aspects. First, Finding 4 shows that perturbations on different types of statements can achieve varying success rates on existing attack techniques. As such, we propose to incorporate such prior knowledge to prioritize identifier selection, thus enhancing the effectiveness and efficiency of the attack. This attack strategy, which incorporates code features, is different from all the previous works, including the SOTA ALERT. Second, Finding 2 reveals that the effectiveness and efficiency of existing attacks are still limited. The reasons are as follows. MHM mainly uses a random method to replace identifiers one by one along the sequence of identifiers that can be replaced while this strategy requires a large number of queries to mislead the model. Meanwhile, ALERT, ACCENT, and WIR-Random replace identifiers sequentially. Once the top candidates fall into local optimal solutions, it is difficult for them to find the global optimal solution since they do not repeatedly process the replaced identifiers. To alleviate such problems, we propose to use *beam search* [12] to focus on all the identifiers in a statement, which can simultaneously search from multiple sequences and replace multiple identifiers.

5.1 Approach Design

Algorithm 1 shows the workflow of BeamAttack. It first obtains the set of identifiers in different statements S and the number of statement types T (Line 2). The priority of different statements is

summarized by our prior knowledge as shown in Table 5. For instance, the prioritized statement types for the clone detection task is: For, If, Try, Method, Throw, Return, and Others. BeamAttack then performs beam search over different types of statements sequentially to generate new examples. For the first iteration, the replaced codes are the source program p (Line 3). For subsequent iterations, the replaced code are the k perturbed codes returned by function BeamSearch (Section 5.1.1) in the previous iteration. The sequence to be replaced (which is denoted as Seq) is initialized to all the identifiers in the entire statement (Line 5). We apply BeamSearch multiple times until the last category of statements is searched or an adversarial example is successfully generated. Note that we record the replaced identifiers after each BeamSearch (Line 7). Finally, BeamAttack performs the last BeamSearch with the recorded replaced identifier as Seq (Line 8). This step is to alleviate the limitation that beam search will not process those identifiers that have already been replaced. Since an identifier can only be replaced by a unique candidate in the adversarial example, some suboptimal candidates are discarded during the search, and they may become optimal after subsequent identifiers are replaced.

5.1.1 BeamSearch. The maximum iteration in the search is set to be the product of Seq 's length and the weight of the statement type (Line 6). In particular, we set the weight of the most important statement type to 1, and the other weights are set proportionally according to the prior knowledge in Table 5. In each iteration in BeamSearch, we apply Perturb (Section 5.1.2) on all the identifiers from the current type of statements. After that, BeamSearch selects the k best ones in the current generation and the previous generation to serve for the next iteration. Note that the search process will stop if the current iteration fails to generate new qualified candidates. Since the number of identifiers is proportional to the maximum number of iterations of BeamSearch, it will directly affect the efficiency of BeamAttack. As shown in Figure 3, the number of identifiers differs for the three tasks. To balance the attack success rate and efficiency, we set k to 2, 3, and 5 in clone detection, vulnerability detection, and code summarization, respectively.

5.1.2 Perturb. Perturb first uses CodeBERT to generate the 30 most similar candidates for an identifier following the four attacks investigated in our study. This similarity is based on the cosine similarity between the embeddings from CodeBERT-MLM following ALERT, which is trained with the objective of masked language modeling. These candidates are generated in real-time for each Perturb. Since some identifiers are changed during the attack, the top 30 candidates for the identifier to be replaced will also change accordingly. Then, Perturb chooses the identifier in the candidate list that reduces the current probability of true label the most for replacement to guarantee the naturalness. If the drop in probability changes the model's predicted label or makes the code summary completely independent of the ground truth (i.e., BLEU = 0), we consider the attack successful and output the adversarial example and the replaced identifier. Otherwise, Perturb returns the perturbed code, original identifier, replaced identifier, and the probability for the corresponding replacement.

5.2 Evaluation

5.2.1 Evaluation Datasets. To evaluate our method thoroughly, we first evaluate it on the dataset as listed in Section 3.1.3. Furthermore,

Algorithm 1: The Main Workflow of BeamAttack

Input: source program p , beam size k , statement weight SW
Output: adversarial example adv

```

1  $rv = []$  # replaced variable
2  $S, T = \text{GetStatements}(p)$  /*  $S$  is the set of identifiers in different statements,  $T$  is the number of statement types */
3  $\mathcal{P}^0 = \{p, S[0]\}$ 
4 for  $t = 0 \rightarrow T$  do
5    $Seq^t = S[t]$ 
6    $max\_iter = \text{Length}(Seq^t) * SW[t]$   $\mathcal{P}^{t+1} \leftarrow \text{BeamSearch}(\mathcal{P}^t, max\_iter)$ 
7    $rv.append(\mathcal{P}^{t+1}.replacedVariable)$ 
8 BeamSearch( $\{Code^T, rv\}, \text{Length}(rv)$ ) Function
   BeamSearch( $\mathcal{P}^t = \{Code^t, Seq^t\}, max\_iter$ )
9   while not exceed  $max\_iter$  do
10     $\mathcal{P}^{copy} \leftarrow \mathcal{P}^t$ 
11    for  $Code_i^t$  in  $\mathcal{P}^t$  do
12      for  $Seq_j^t$  in  $\mathcal{P}^t$  do
13         $\mathcal{P}_i^{t+1} \leftarrow \text{Perturb}(Code_i^t, Seq_j^t)$ 
14       $\mathcal{P}^t \leftarrow \text{Selection}(\mathcal{P}^t \cup \mathcal{P}^{t+1}, k)$ 
15      if  $\mathcal{P}^t == \mathcal{P}^{copy}$  then
16        return  $\mathcal{P}^t$ 
17   return  $\mathcal{P}^t$ 

```

to demonstrate its generalizability, we perform additional evaluations on three other datasets. For clone detection, we use the filtered Google Code Jam (GCJ) [31, 42] dataset consisting of 90,102 examples for training and 4,000 for validation and testing respectively. For vulnerability detection, we use the Juliet Test Suite², which is another widely used open source security benchmark [25, 35] besides OWASP. In particular, we utilize the Java sub-dataset and exclude instances with identical function bodies, and finally obtain the training, validation, and testing sets consisting of 23,636, 2,954, and 2,954 examples respectively. For code summarization, we use TL-CodeSum (TLC) [10], which is widely used as a benchmark [1, 24]. Similar to Juliet, we filter out duplicate examples and obtain the training, validation, and testing sets consisting of 69,633, 8,700, and 6,445 examples respectively. We also try to reproduce the results on these datasets, and the results reflect that our fine-tuned models can also achieve similar performance.

5.2.2 Evaluation Results. We evaluate BeamAttack and ALERT on 18 sets of experiments (3 pre-trained models $\times$ 3 tasks $\times$ 2 datasets) as they both use the context-aware identifier prediction as the substitution strategy to guarantee the naturalness of adversarial examples. The evaluation results are summarized in Table 6, and we can observe that BeamAttack consistently achieves higher attack success rates on all experiments. On average, BeamAttack outperforms ALERT by 20.85% in terms of ASR, showing that it is more effective in achieving successful attacks. With respect to the attack efficiency, BeamAttack outperforms ALERT on 15/18 experiments. In addition, the average AMQ of BeamAttack is 1,690.12, which is 11.98% less than that of ALERT (1,920.18 on average). This indicates that our method, which relies on the statement importance to search for adversarial examples, is more efficient. Since BeamAttack replaces identifiers based on context-aware identifier prediction, the adversarial examples generated by it are of higher qualities with lower perturbation rates. Specifically, the average ICR of BeamAttack is 8.49, which is lower than that of ALERT (11.32 on average). Meanwhile, the average ICR and TCR of BeamAttack on the 18

²<https://samate.nist.gov/SARD/test-suites>

Table 6: Performance comparison between BeamAttack and ALERT

Attack Results		Clone Detection-BCB							Vulnerability Detection-OWASP							Code Summarization-CSN						
		ASR↑	AMQ↓	ART↓	ICR↓	TCR↓	ACS↑	AED↓	ASR↑	AMQ↓	ART↓	ICR↓	TCR↓	ACS↑	AED↓	ASR↑	AMQ↓	ART↓	ICR↓	TCR↓	ACS↑	AED↓
CodeBERT	BeamAttack	22.43	1,992.97	2.69	9.06	1.49	0.9872	66.45	5.04	1,956.58	2.94	2.90	0.43	0.9815	94.06	51.46	371.82	1.53	15.05	1.80	0.9951	15.91
	ALERT	14.48	2,263.53	4.49	8.77	1.00	0.9837	78.35	4.23	2,718.57	2.02	2.79	0.34	0.9868	77.72	48.58	565.63	2.30	23.68	2.61	0.9907	23.63
CodeGPT	BeamAttack	12.09	2,364.90	3.40	7.42	1.18	0.9800	106.91	6.11	1,904.41	3.02	3.50	0.56	0.9784	104.26	23.86	400.26	4.32	7.20	0.84	0.9948	15.71
	ALERT	6.59	2,596.18	5.89	4.72	0.57	0.9862	96.48	4.75	2,798.29	1.55	3.03	0.39	0.9829	89.23	23.50	938.39	8.78	13.37	1.52	0.9879	29.01
PLBART	BeamAttack	13.14	2,327.70	5.75	6.02	0.99	0.9856	76.76	19.17	1,742.51	3.40	7.05	1.15	0.9861	65.79	58.29	361.22	1.19	18.21	2.26	0.9948	16.29
	ALERT	9.10	2,549.49	8.39	6.26	0.69	0.9832	79.94	17.28	2,592.32	3.65	9.67	1.23	0.9844	73.10	54.37	488.82	1.70	29.55	3.36	0.9891	25.90
Attack Results		Clone Detection-GCJ							Vulnerability Detection-Juliet							Code Summarization-TLC						
		ASR↑	AMQ↓	ART↓	ICR↓	TCR↓	ACS↑	AED↓	ASR↑	AMQ↓	ART↓	ICR↓	TCR↓	ACS↑	AED↓	ASR↑	AMQ↓	ART↓	ICR↓	TCR↓	ACS↑	AED↓
CodeBERT	BeamAttack	11.84	4,702.12	6.38	4.52	1.02	0.9795	82.00	0.17	1,176.07	0.95	0.10	0.01	0.9868	85.40	44.23	602.48	4.90	13.35	2.07	0.9924	21.52
	ALERT	6.85	4,008.96	2.86	4.13	0.80	0.9739	95.19	0.14	1,872.35	1.78	0.05	0.00	0.9837	30.50	31.87	516.50	6.54	14.13	1.98	0.9877	26.63
CodeGPT	BeamAttack	25.72	3,147.57	5.73	6.60	1.65	0.9861	55.97	0.85	1,554.57	1.21	0.75	0.10	0.9877	139.88	58.58	255.56	1.69	13.42	1.76	0.9948	13.70
	ALERT	20.42	3,170.36	2.77	8.70	1.53	0.9813	72.60	0.27	1,847.87	1.92	0.22	0.02	0.9889	92.00	52.66	603.25	6.44	19.90	2.52	0.9904	20.31
PLBART	BeamAttack	47.32	3,446.06	4.96	13.45	3.31	0.9851	75.85	1.09	1,747.10	2.53	0.56	0.07	0.9913	74.62	67.29	368.27	2.40	17.78	2.44	0.9945	15.15
	ALERT	36.67	2,537.58	3.20	19.54	3.71	0.9794	98.24	0.78	1,973.91	2.28	0.45	0.04	0.9953	56.48	55.27	521.31	6.22	23.79	3.37	0.9889	23.36

Note: Bold numbers indicate the better performance for the given metric. The cell with lightgray background denotes the outperformance is significant ($p < 0.05$).

```

1 public static void main(String args[]) {
2     Scanner scanner = new Scanner(new
3         FileReader(inFile));
4     int T = scanner.nextInt();
5     FileWriter fw=new FileWriter(outFile);
6     for (int t = 1; t <= T; ++t) {
7         int r = scanner.nextInt();
8         int c = scanner.nextInt();
9         int w = scanner.nextInt();
10        fw.write(String.format("%d:%s",
11            t,solve(r,c,w)));
12    }
13 }
14 private static int solve(int r,int c,int w){
15     int res=((c-1)/w+1)*(r-1)+w+(c-1)/w;
16     return res; }

```

(a) Clone Detection: the first version of index #1216 from dataset GCJ

```

1 public static void main(String[] args){
2     BufferedReader br = new BufferedReader(
3         new InputStreamReader(System.in));
4     String str = br.readLine();
5     while ((str = br.readLine()) != null) {
6         String[] temp = str.split(" ");
7         int r = Integer.parseInt(temp[0]);
8         int c = Integer.parseInt(temp[1]);
9         int w = Integer.parseInt(temp[2]);
10        int ans = 0;
11        if (c % w == 0) {
12            ans = ((c / w) + w - 1) * r;
13        } else {
14            ans = ((c / w) + w) * r;
15        }
16    }

```

(b) Clone Detection: the cloned version of index #1216 from dataset GCJ

```

1 Cookie[] cookies=request.getCookies();
2 String param = null;
3 boolean foundit = false;
4 if (cookies != null) {
5     for (Cookie cookie:cookies) {
6         if (cookie.getName().equals("foo")){
7             param = cookie.getValue();
8             foundit = true; } }
9     if (!foundit) { param = ""; }
10 } else {
11     param = "";
12 }
13 String bar = new Test().doSomething(param);
14 java.io.FileOutputStream fos = new java.io.
15     FileOutputStream(org.owasp.benchmark.
16         helpers.Utils.testfileDir + bar,false);

```

(c) Vulnerability Detection: Index #26 from dataset OWASP

Figure 5: Case study on clone detection and vulnerability detection

experiments are 8.16 and 1.29, which are lower than ALERT (10.71 and 1.43, respectively). The average ACS of BeamAttack is 0.9871, which outperforms ALERT (0.9861). BeamAttack only performs worse than ALERT on AED (62.57 vs. 60.54).

To verify whether the performance differences are statistically significant, we apply the one-sided Mann-Whitney U tests [21] to each experiment. Significant differences (p -value <0.05) are marked with lightgray background in Table 6. The results show that our method outperforms ALERT on 69.05% of the cases (i.e., 87/126, 18 experiments * 7 metrics). Among these cases, 78 (accounting for 89.66%) demonstrate significant differences, strongly verifying that our method not only surpasses ALERT on most evaluation metrics, but also achieves significant outperformance. Although ALERT performs relatively better in the remaining 39 cases, we note the difference between BeamAttack and ALERT is significant for only 22 cases.

6 DISCUSSION

6.1 Case Study

In this section, we perform an additional case study to qualitatively compare BeamAttack with ALERT to understand their distinctions and why prioritizing statements can significantly affect the performance of adversarial attacks. We illustrate based on Figure 5 on clone detection and vulnerability detection. The cloned code corresponding to Figure 5a transforms `for` loop into `while` and refactors the `solve` function, as shown in Figure 5b. Figure 5c displays a code snippet from the OWASP dataset that contains a *Path Traversal* vulnerability on lines 14-16. The vulnerability exists due to the lack of input validation for the variable `bar` which receives data from the `cookie` (i.e., `param`). Such validation is often performed by using

the `if` statement, which highlights the importance of `if` in influencing the predictions for vulnerability detection models. These two cases can reflect our finding in RQ3 that *the statements have the most significant impact on adversarial attacks against clone detection and vulnerability detection are For and If, respectively*. Specifically, during the attack process, the first set of identifiers extracted by BeamAttack are: [For: [fw, r, c, t, T, w, scanner]] and [If: [cookie, foundit, i, cookies, bar, param]] respectively since it prioritizes those statements based on the learned prior knowledge. One successful attack substitution follows [w→j, c→k, fw→ww] and [cookies→Cooks, param→ram, bar→ban, i→vi, foundit→foundait, doSomething→runNothing] respectively. It can be seen that successful attacks can be achieved by only replacing part of the identifiers in the For and If statements. In contrast, the sequences replaced by ALERT are: [c, fw, w, res, r, inFile, outFile, t, T, scanner] and [bar, param, response, foundit, fos, i] respectively. We note that ALERT does not accurately replace the identifiers required to achieve the attack. For example, it replaces irrelevant identifiers `inFile` and `outFile` in clone detection, and also misses the identifier `cookies` that is highly relevant to the vulnerability. The above analysis explains the distinction between ALERT and BeamAttack as well as why ALERT is less effective.

In addition to the difference in prioritizing identifier substitution, it is worth noting that we also utilize beam search to attack, which allows more sequences for identifier substitutions within the same statement. For instance, in Figure 5c, ALERT replaces identifiers strictly according to a fixed order, meaning only `param` can come after `bar`. In contrast, after replacing `bar`, BeamAttack can choose `foundit`, `param`, or `i` as the next replaceable identifier, which mitigates the risk of getting stuck in local solutions.

6.2 Implications

For model robustness, we find that existing PTMCs are susceptible to adversarial attacks (Finding 1), which presents a considerable challenge to their robustness. Therefore, we strongly advocate that researchers need to place equal, if not greater, emphasis on improving model robustness while striving to improve model performance. Practical strategies such as adversarial training or data augmentation can be employed to enhance robustness. These methods can equip models with the resilience required to counter adversarial perturbations and improve their overall reliability.

For adversarial attack approach, considering the trend of *large language models* (LLMs) towards being closed-source and chargeable, attacks through massive queries may become extremely costly or even infeasible. Therefore, a further exploration of the relationship between code identifiers and model prediction results is necessary to derive a more accurate sequence for identifier replacement. Additionally, we can leverage the SOTA language generation techniques, such as ChatGPT, to replace identifiers and enhance the naturalness of adversarial examples. Finally, considering the effectiveness of statement prioritization, future methods should focus on a more in-depth analysis of code structure, such as employing comprehensive semantic analysis to devise more effective attacks.

7 THREATS TO VALIDITY

Internal validity: Parameter settings such as the number of iterations can lead to different results. We adopt the following strategy to mitigate this threat. When the parameters can be set uniformly, we set the parameters consistently with the five attacks, such as the number of candidates for the identifier. When the parameters are specific to an attack approach, we follow the settings in the original paper exactly to achieve fair comparisons. Another internal validity threat is the potential bugs in our implementation. To reduce such threats, we have carefully checked our implementations and also open sourced all the materials and code to the community for further checks.

External validity: External validity is threatened by the generalizability of tasks, datasets, and models. For tasks, the selected ones have been extensively studied in existing works on adversarial attacks [34, 43]. For datasets, we not only use the CodeXGLUE benchmark studied in many original papers on PTMCs and adversarial attacks, but also include a new dataset, the OWASP benchmark, to evaluate the general applicability of the attack approaches. For the target models, we mitigate this threat by selecting the most popular PTMCs with relatively high performance.

8 RELATED WORK

Black box attack approaches have been extensively discussed in Section 3.1.2, and we introduce other *white box attack* approaches in this section. Specifically, Yefet *et al.* [36] propose DAMP, which utilizes the gradient information of the target model to find replacement identifiers in the opposite direction of the gradient descent. Meanwhile, they use *one-hot vector* to encode code, aiming at obtaining candidates by perturbing the vector and then mapping them back to tokens. However, such an approach is unable to constrain the candidates so that it may obtain irrelevant identifiers similar to random substitutions. Srikanth *et al.* [26] turn the adversarial attack into an optimization problem, and identify two aspects in

the adversarial attack: which parts of the program to transform, and what transformations to use. They correspond to the search strategy and replacement strategy respectively as we mentioned above. Then, they use *projected gradient descent* (PGD) based *joint optimization* (JO) solver to obtain the optimal transform location and transform method. Zhang *et al.* [39] propose CARROT, which incorporates gradient information into transform operations to guide the search process more effectively. Although retrieving gradients during transform operations may take more time, it can effectively reduce search iterations. The above white box attacks are not very practical as the latest SOTA models, such as ChatGPT, are increasingly becoming closed-source. These models are typically deployed remotely and offer services through API interfaces, making it difficult to access their internal structure and parameters.

There is no comprehensive evaluation towards adversarial attack on PTMCs currently, and the study most similar to ours is that of Zeng *et al.* [38]. However, they mainly focus on evaluating the effectiveness of PTMCs while the adversarial attack approaches are not fully studied. Specifically, they evaluate several attack approaches adapted from the field of *natural language processing* (NLP), and focus on comparison in terms of ASR. In contrast, our study specifically focuses on attacks designed for SE applications, and we have additionally evaluated attack efficiency and the quality of the generated adversarial examples. Moreover, the attack approach they proposed simply combines WIR and random replacement without incorporating the unique characteristics of programming languages and code intelligence tasks, which is a common shortfall for most existing studies on adversarial code attacks. In this work, for the first time, we generate adversarial examples by perturbing source code based on the contextual information of identifiers.

9 CONCLUSION

This study thoroughly evaluates the performance, efficiency, and robustness of adversarial attacks on PTMCs. Results show that PTMCs are easily susceptible to adversarial perturbations, with varying levels of robustness among different tasks. The code summarization model is found to be the most vulnerable. Additionally, high-performing attack approaches often come with significant computational overhead. The importance of different statements is also analyzed, revealing varying levels of sensitivity among different context identifiers to counterattacks. Based on such findings, we propose a new approach, BeamAttack, which improves the effectiveness of attacks by 21.30% and efficiency by 14.62% compared to the existing approach ALERT using the same identifier substitution strategy.

10 DATA AVAILABILITY

The data, source code, and the results of this paper are available at: https://github.com/CGCL-codes/Attack_PTMC.

ACKNOWLEDGMENTS

We sincerely thank all anonymous reviewers for their valuable comments. This work was supported by the Key Program of Hubei under Grant No. 2023BAA024, the National Natural Science Foundation of China (Grant No. 62002125), and the Young Elite Scientists Sponsorship Program by CAST (Grant No. 2021QNRC001).

REFERENCES

- [1] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2020. A Transformer-based Approach for Source Code Summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel R. Tetreault (Eds.). Association for Computational Linguistics, 4998–5007. <https://doi.org/10.18653/v1/2020.acl-main.449>
- [2] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified Pre-training for Program Understanding and Generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, 2655–2668. <https://doi.org/10.18653/v1/2021.naacl-main.211>
- [3] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [4] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, 7212–7225. <https://doi.org/10.18653/v1/2022.acl-long.499>
- [5] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *Proceedings of the 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=jLoC4ez43PZ>
- [6] Xu Han, Zhengyan Zhang, Ning Ding, Yuxian Gu, Xiao Liu, Yuqi Huo, Jiezhong Qiu, Yuan Yao, Ao Zhang, Liang Zhang, Wentao Han, Minlie Huang, Qin Jin, Yanyan Lan, Yang Liu, Zhiyuan Liu, Zhiwu Lu, Xipeng Qiu, Ruihua Song, Jie Tang, Ji-Rong Wen, Jinhui Yuan, Wayne Xin Zhao, and Jun Zhu. 2021. Pre-trained models: Past, present and future. *AI Open* 2 (2021), 225–250. <https://doi.org/10.1016/j.aiopen.2021.08.002>
- [7] Jordan Henkel, Goutham Ramakrishnan, Zi Wang, Aws Albarghouthi, Somesh Jha, and Thomas W. Reps. 2022. Semantic Robustness of Models of Source Code. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2022, Honolulu, HI, USA, March 15-18, 2022*. IEEE, 526–537. <https://doi.org/10.1109/SANER53432.2022.00070>
- [8] Christopher Herbig. 2002. Genetic Algorithms vs. Greedy Algorithms in the Optimization of Course Scheduling. *J. Comput. Sci. Coll.* 17, 5 (apr 2002), 90–94. <https://dl.acm.org/doi/pdf/10.5555/775009.775028>
- [9] Katherine Hough, Gebrehiet B. Welearegai, Christian Hammer, and Jonathan Bell. 2020. Revealing injection vulnerabilities by leveraging existing tests. In *Proceedings of the 42nd International Conference on Software Engineering, ICSE 2020, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 284–296. <https://doi.org/10.1145/3377811.3380326>
- [10] Xing Hu, Ge Li, Xin Xia, David Lo, Shuai Lu, and Zhi Jin. 2018. Summarizing Source Code with Transferred API Knowledge. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI 2018, July 13-19, 2018, Stockholm, Sweden, Jérôme Lang (Ed.)*. ijcai.org, 2269–2275. <https://doi.org/10.24963/ijcai.2018/314>
- [11] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *CoRR abs/1909.09436* (2019). arXiv:1909.09436 <http://arxiv.org/abs/1909.09436>
- [12] Abhishek Kumar, Shankar Vembu, Aditya Krishna Menon, and Charles Elkan. 2013. Beam search algorithms for multilabel learning. *Mach. Learn.* 92, 1 (2013), 65–89. <https://doi.org/10.1007/s10994-013-5371-6>
- [13] Zhen Li, Qian (Guenevere) Chen, Chen Chen, Yayi Zou, and Shouhuai Xu. 2022. RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1906–1918. <https://doi.org/10.1145/3510003.3510181>
- [14] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round1.html>
- [15] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. 1953. Equation of state calculations by fast computing machines. *The journal of chemical physics* 21, 6 (1953), 1087–1092. <https://doi.org/10.1063/1.1699114>
- [16] Phuong T. Nguyen, Claudio Di Sipio, Juri Di Rocco, Massimiliano Di Penta, and Davide Di Ruscio. 2021. Adversarial Attacks to API Recommender Systems: Time to Wake Up and Smell the Coffee. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*. IEEE, 253–265. <https://doi.org/10.1109/ASE51524.2021.9678946>
- [17] Maryam Vahdat Pour, Zhuo Li, Lei Ma, and Hadi Hemmati. 2021. A Search-Based Testing Framework for Deep Neural Networks of Source Code Embedding. In *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation, ICST 2021, Porto de Galinhas, Brazil, April 12-16, 2021*. IEEE, 36–46. <https://doi.org/10.1109/ICST49551.2021.00016>
- [18] Dong Qiu, Bixin Li, Earl T. Barr, and Zhendong Su. 2017. Understanding the syntactic rule usage in java. *J. Syst. Softw.* 123 (2017), 160–172. <https://doi.org/10.1016/j.jss.2016.10.017>
- [19] Xipeng Qiu, Tianxiang Sun, Yige Xu, Yunfan Shao, Ning Dai, and Xuanjing Huang. 2020. Pre-trained Models for Natural Language Processing: A Survey. *CoRR abs/2003.08271* (2020). arXiv:2003.08271 <https://arxiv.org/abs/2003.08271>
- [20] Md. Rafiqul Islam Rabin, Nghi D. Q. Bui, Ke Wang, Yijun Yu, Lingxiao Jiang, and Mohammad Amin Alipour. 2021. On the generalizability of Neural Program Models with respect to semantic-preserving program transformations. *Inf. Softw. Technol.* 135 (2021), 106552. <https://doi.org/10.1016/j.infsof.2021.106552>
- [21] Graeme D. Ruxton. 2006. The unequal variance t-test is an underused alternative to Student's t-test and the Mann-Whitney U test. *Behavioral Ecology* 17, 4 (05 2006), 688–690. <https://doi.org/10.1093/beheco/ark016>
- [22] Mohammadreza Salehi, Hossein Mirzaei, Dan Hendrycks, Yixuan Li, Mohammad Hossein Rohban, and Mohammad Sabokrou. 2022. A Unified Survey on Anomaly, Novelty, Open-Set, and Out of-Distribution Detection: Solutions and Future Challenges. *Trans. Mach. Learn. Res.* 2022 (2022). <https://openreview.net/forum?id=aRtjVZvbpK>
- [23] Imen Sayar, Alexandre Bartel, Eric Bodden, and Yves Le Traon. 2023. An In-depth Study of Java Deserialization Remote-Code Execution Exploits and Vulnerabilities. *ACM Trans. Softw. Eng. Methodol.* 32, 1 (2023), 25:1–25:45. <https://doi.org/10.1145/3554732>
- [24] Ensheng Shi, Yanlin Wang, Lun Du, Junjie Chen, Shi Han, Hongyu Zhang, Dongmei Zhang, and Hongbin Sun. 2022. On the Evaluation of Neural Code Summarization. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1597–1608. <https://doi.org/10.1145/3510003.3510060>
- [25] Fausto Spoto, Elisa Burato, Michael D. Ernst, Pietro Ferrara, Alberto Lovato, Damiano Macedonio, and Ciprian Spiridon. 2019. Static Identification of Injection Attacks in Java. *ACM Trans. Program. Lang. Syst.* 41, 3 (2019), 18:1–18:58. <https://doi.org/10.1145/3332371>
- [26] Shashank Srikant, Sijia Liu, Tamara Mitrovska, Shiyu Chang, Quanfu Fan, Gaoyuan Zhang, and Una-May O'Reilly. 2021. Generating Adversarial Computer Programs using Optimized Obfuscations. In *Proceedings of the 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. https://openreview.net/forum?id=PH5PH9ZO_4
- [27] Jeffrey Svajlenko, Judith F. Islam, Iman Keivanloo, Chanchal Kumar Roy, and Mohammad Mamun Mia. 2014. Towards a Big Data Curated Benchmark of Inter-project Code Clones. In *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution, Victoria, BC, Canada, September 29 - October 3, 2014*. IEEE Computer Society, 476–480. <https://doi.org/10.1109/ICSME.2014.77>
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Proceedings of the 31st Annual Conference on Neural Information Processing Systems, NIPS 17, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [29] Andrew Walker, Tomas Cerny, and Eungh Song. 2020. Open-Source Tools and Benchmarks for Code-Clone Detection: Past, Present, and Future Trends. *SIGAPP Appl. Comput. Rev.* 19, 4 (jan 2020), 28–39. <https://doi.org/10.1145/3381307.3381310>
- [30] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. 2022. No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.). ACM, 382–394. <https://doi.org/10.1145/3540250.3549113>

- [31] Wenhan Wang, Ge Li, Bo Ma, Xin Xia, and Zhi Jin. 2020. Detecting Code Clones with Graph Neural Network and Flow-Augmented Abstract Syntax Tree. In *Proceedings of the 27th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2020, London, ON, Canada, February 18–21, 2020*, Kostas Kontogiannis, Foutse Khomh, Alexander Chatzigeorgiou, Marios-Eleftherios Fokaefs, and Minghui Zhou (Eds.). IEEE, 261–271. <https://doi.org/10.1109/SANER48275.2020.9054857>
- [32] Wenqi Wang, Run Wang, Lina Wang, Zhibo Wang, and Aoshuang Ye. 2023. Towards a Robust Deep Neural Network Against Adversarial Texts: A Survey. *IEEE Trans. Knowl. Data Eng.* 35, 3 (2023), 3159–3179. <https://doi.org/10.1109/TKDE.2021.3117608>
- [33] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7–11 November, 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, 8696–8708. <https://doi.org/10.18653/v1/2021.emnlp-main.685>
- [34] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural Attack for Pre-trained Models of Code. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25–27, 2022*. ACM, 1482–1493. <https://doi.org/10.1145/3510003.3510146>
- [35] Jiayi Ye, Chao Zhang, and Xinhui Han. 2014. POSTER: UAFChecker: Scalable Static Detection of Use-After-Free Vulnerabilities. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3–7, 2014*, Gail-Joon Ahn, Moti Yung, and Ninghui Li (Eds.). ACM, 1529–1531. <https://doi.org/10.1145/2660267.2662394>
- [36] Noam Yefet, Uri Alon, and Eran Yahav. 2020. Adversarial examples for models of code. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 162:1–162:30. <https://doi.org/10.1145/3428230>
- [37] Guoyang Zeng, Fanchao Qi, Qianrui Zhou, Tingji Zhang, Zixian Ma, Bairu Hou, Yuan Zang, Zhiyuan Liu, and Maosong Sun. 2021. OpenAttack: An Open-source Textual Adversarial Attack Toolkit. In *Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL 2021 - System Demonstrations, Online, August 1–6, 2021*, Heng Ji, Jong C. Park, and Rui Xia (Eds.). Association for Computational Linguistics, 363–371. <https://doi.org/10.18653/v1/2021.acl-demo.43>
- [38] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022, Virtual Event, South Korea, July 18 – 22, 2022*, Sukyoung Ryu and Yannis Smaragdakis (Eds.). ACM, 39–51. <https://doi.org/10.1145/3533767.3534390>
- [39] Huangzhao Zhang, Zhiyi Fu, Ge Li, Lei Ma, Zhehao Zhao, Hua'an Yang, Yizhe Sun, Yang Liu, and Zhi Jin. 2022. Towards Robustness of Deep Program Processing Models - Detection, Estimation, and Enhancement. *ACM Trans. Softw. Eng. Methodol.* 31, 3 (2022), 50:1–50:40. <https://doi.org/10.1145/3511887>
- [40] Huangzhao Zhang, Zhuo Li, Ge Li, Lei Ma, Yang Liu, and Zhi Jin. 2020. Generating Adversarial Examples for Holding Robustness of Source Code Processing Models. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, New York, NY, USA, February 7–12, 2020*. AAAI Press, 1169–1176. <https://ojs.aaai.org/index.php/AAAI/article/view/5469>
- [41] Wei Emma Zhang, Quan Z. Sheng, Ahoud Alhazmi, and Chenliang Li. 2020. Adversarial Attacks on Deep-learning Models in Natural Language Processing: A Survey. *ACM Trans. Intell. Syst. Technol.* 11, 3 (2020), 24:1–24:41. <https://doi.org/10.1145/3374217>
- [42] Gang Zhao and Jeff Huang. 2018. DeepSim: deep learning code functional similarity. In *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, USA, November 04–09, 2018*, Gary T. Leavens, Alessandro Garcia, and Corina S. Pasareanu (Eds.). ACM, 141–151. <https://doi.org/10.1145/3236024.3236068>
- [43] Yu Zhou, Xiaoqing Zhang, Juanjuan Shen, Tingting Han, Taolue Chen, and Harald C. Gall. 2022. Adversarial Robustness of Deep Code Comment Generation. *ACM Trans. Softw. Eng. Methodol.* 31, 4 (2022), 60:1–60:30. <https://doi.org/10.1145/3501256>

Received 2023-03-02; accepted 2023-07-27